

第一章：应用程序与基本执行环境

引言

本章导读

本章展现了操作系统一个功能：让应用与硬件隔离，简化了应用访问硬件的难度和复杂性。

大多数程序员的第一行代码都从 `Hello, world!` 开始，当我们满怀好奇心在编辑器内键入仅仅数个字节，再经过几行命令编译（靠的是编译器）、运行（靠的是操作系统），终于在黑洞洞的终端窗口中看到期望中的结果的时候，一扇通往编程世界的大门已经打开。在本章第一节 [代码框架简述](#) 中，可以看到用C语言编写的非常简单的"Hello, world"应用程序。

不过我们能够隐约意识到编程工作能够如此方便简洁并不是理所当然的，实际上有着多层硬件和软件工具和支撑环境隐藏在它背后，才让我们不必付出那么多努力就能够创造出功能强大的应用程序。生成应用程序二进制执行代码所依赖的是以 **编译器** 为主的开发环境；运行应用程序执行码所依赖的是以 **操作系统** 为主的执行环境。

本章我们将从操作系统最简单但也是最重要的println入手，要求大家实现一个裸机上的println以及带色彩的LOG，如info和warn，error等功能。因为大家是刚刚接触操作系统实验，本章的所有代码已经帮大家写好了，没有大家需要亲自编写代码的部分。但是它作为第一章又是最重要的一个章节：这一章之中，同学们要对整个C的OS实验框架有一个大致的掌握。对整个框架是如何编译的，之后需要写哪些内容以及如何测试有一个基本的认识。可以说，ch1打好基础会使得之后的实验难度大大降低。

系统调用

在实验开始之前，大家要熟悉一下系统调用(syscall)的概念。相信大家在汇编的课程中一定接触过这个名词。我们OS课程中的syscall的意义也是一样的，它是操作系统提供给软件的一系列接口，使得软件能够使用系统的功能。syscall本质上属于一种异常/中断，它在riscv的汇编指令中以ecall的形式出现。

本章的println所需要的在console中打印字符，也需要调用到syscall。syscall的种类有很多，操作系统通过区分syscall的id来判断是哪一个syscall。

实践体验

获取本章代码：

```
$ git checkout ch1
```

在qemu模拟器上运行本章代码，看看一个小应用程序是如何在QEMU模拟的计算机上运行的：

```
$ make run LOG=trace
```

警告

FIXME: 提供 wsl/macOS 等更多平台支持

如果顺利的话，以qemu平台为例，将输出：

 ch1-demo.png

除了 `Hello, world!` 之外还有一些额外的信息，最后关机。

注解

RustSBI是啥？

戳 [附录 C：深入机器模式：RustSBI](#) 可以进一步了解RustSBI。

展望未来

现在我们的 os 会直接关机，在我们完成 lab5 之后，我们的 os 就可以比较自由的运行用户程序了。具体来说，我们会会有一个 shell。



代码框架简述

本节导读

本节会介绍代码的整体框架。我们默认大家都熟练掌握了C语言。整个项目目前的代码树如下：

```
.
├── bootloader
│   └── rustsbi-qemu.bin
├── LICENSE
├── Makefile
├── os
│   ├── console.c
│   ├── console.h
│   ├── defs.h
│   ├── entry.S
│   ├── kernel.ld
│   ├── log.h
│   ├── main.c
│   ├── printf.c
│   ├── printf.h
│   ├── riscv.h
│   ├── sbi.c
│   ├── sbi.h
│   └── types.h
└── README.md
```

OS是怎么跑起来的？

我们的OS的运行，是要依赖著名的模拟器软件-qemu的。比较形象的比喻是，我们的os就是一个内核软件，qemu就类似一个主板，它模拟了许多硬件，比如CPU，I/O串口等等。我们的OS会和qemu模拟出来的这些硬件打交道，而qemu则把得到的指令分配给实际存在的硬件完成。

我们的OS启动的时候，就像一个真正的操作系统启动一样。qemu使用我们提供的rustsbi的bin文件做为引导程序来启动OS。同时，我们的内核做为运行在qemu中的虚拟机，是无法直接和我们的外部host系统通信的，因此我们OS自己实现的printf函数，想要真正地输出到我们外部运行的shell上被我们看到，是要经过qemu的。实际上，在启动时sbi已经帮我们初始化好了，经过qemu模拟出来的串口，最终打印到我们外部的shell上的。之后，从我们的shell之中读取输入，也是同样的道理。sbi为我们内核提供的功能不止于输入输出，在sbi.c文件的可以看到其他支持的功能，比如关机。

注解

RustSBI 是什么？

SBI 是 RISC-V 的一种底层规范，RustSBI 是它的一种实现。操作系统内核与 RustSBI 的关系有点像应用与操作系统内核的关系，后者向前者提供一定的服务。只是SBI提供的服务很少，比如关机，显示字符串，读入字符串等。

qemu是怎么跑起来的？

qemu 拓展阅读: [qemu参数](#)。

第0章大家配置好了qemu之后可能就没再打开过了。qemu做为模拟器用途很多，操作也比较复杂。因此我们在makefile之中提供了具体运行qemu所需要的参数，大家无需更改。

```
QEMU = qemu-system-riscv64
QEMUOPTS = "
    -nographic "
    -smp $(CPUS) "
    -machine virt "
    -bios $(BOOTLOADER) "
    -kernel kernel

run: $(BUILDDIR)/kernel
    $(QEMU) $(QEMUOPTS)
```

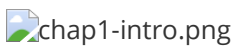
这个就是最关键的地方:make run。我们查看这条指令的结构，它首先执行上面 kernel 所需要的链接以及编译操作得到一个二进制的kernel。之后执行按照QEMUOPTS变量指定的参数启动qemu。QEMUOPTS意义如下：

- nographic: 无图形界面
- smp 1: 单核 (默认值，可以省略)
- machine virt: 模拟硬件 RISC-V VirtIO Board
- bios \$(bios): 使用制定 bios，这里指向的是我们提供的 rustsbi 的bin文件。
- kernel: 使用 elf 格式的 kernel。这里就是我们需写的OS内核了。

警告

每次在make run之前，尽量先执行make clean以删除缓存，特别是在切换ch分支之后。

make run这个指令，应该会陪伴大家走过接下来所有的实验qaq。它完成了内核代码的编译生成kernel，并按照QEMUOPTS变量指定的参数加载我们的kernel，"加电"启动qemu。此时，CPU 的其它通用寄存器清零，而 PC 会指向 0x1000 的位置，这里有固化在硬件中的一小段引导代码，它会很快跳转到 0x80000000 的 RustSBI 处。RustSBI 完成硬件初始化后，会跳转到 \$(KERNEL_BIN) 所在内存位置 0x80200000 处，执行我们操作系统的第一条指令。



那么，知道了这些步骤之后，关键就是怎么去写我们的OS了，这也是我们接下来各个实验的内容~。我们OS的代码，基本全部在os文件夹下。nfs文件夹下有一些文件系统相关的内容，在第七章之前大家无需关注这个文件夹下的内容。

os文件夹

os文件夹下存放了我们构建操作系统的源代码，是本次实验中最最重要的一部分，也是整个实验过程中同学们唯一需要修改的部分。在开始实验之前，大家一定要清楚我们这是自己设计的 OS，是无法使用C提供的官方标准库的，也就是说，就算是最简单的 printf 之类的函数都无法使用。还好，作为一个轻量级的 OS，我们也用不到那么多函数。

我们的os是一个由makefile来构建的C项目。下面介绍框架之中一些重要文件的作用，以及整个项目是如何链接及编译的。

- **kernel.ld**

kernel.ld是我们用于链接项目的脚本。链接脚本决定了 elf 程序的内存空间布局(严格的讲是虚存映射，注意程序中的各种绝对地址就在链接的时候确定)，由于刚进入 S 态的时候我们尚未激活虚存机制，我们必须把 os 置于物理内存的 0x80200000 处（这个地址的来由请参考 rustsbi）

```
BASE_ADDRESS = 0x80200000;
SECTIONS
{
    . = BASE_ADDRESS;
    skernel = .;

    stext = .;
    .text : {
        *(.text.entry)    # 第一行代码
        *(.text .text.*)
    }

    ...
}
```

SECTIONS 之中是从 BASE_ADDRESS 开始的各段。对程序内存布局还不太熟悉的同学可以翻看后面内存布局的章节。以 text 段为例，它是由不同文件的 text 组成。我们没有规定这些 text 段的具体顺序，但是我们规定了一个特殊的 text 段：.text.entry 段，该 text 段是 BASE_ADDRESS 后的第一个段，该段的第一行代码就在 0x80200000 处。这个特殊的段不是编译生成的，它在 entry.S 中人为设定。

- **entry.S**

```
# entry.S

.section .text.entry
.global _entry
_entry:
    la sp, boot_stack
    call main

.section .bss.stack
.global boot_stack
boot_stack:
    .space 4096 * 16
    .global boot_stack_top
boot_stack_top:
```

.text.entry 段中只有一个函数 _entry，它干的事情也十分简单，设置好 os 运行的堆栈（la sp, boot_stack 语句。bootloader 并没有好心的设置好这些），然后调用 main 函数。main 函数位于 main.c 中，从此开始我们就基本进入了 C 的世界。

- **main.c**

它是os的入口函数。在其中我们会完成一系列的初始化并开始运行os。作为第一章，它在初始化完毕之后实际上起到了一个测试的作用。如果你的main.c能够完成一系列打印并且最后成功退出(Shutdown),那么祝贺你，你完成了os的第一步。

```
extern char s_text[];
extern char e_text[];

// ...

void main()
{
    clean_bss();
    console_init();
    printf("\n");
    printf("hello world!\n");
    errorf("stext: %p", s_text);
    // ...
    errorf("ebss : %p", e_bss);
    panic("ALL DONE");
}
```

其中，main.c 之中众多的 extern 声明的内存段是在 ld 文件之中定义的，通过这些 symbol 我们可以大致了解 OS 的内存布局。

此外 `clean_bss()` 清空了 `bss` 段，注意，清空 elf 程序 `.bss` 段这一工作通常是由 OS 做的，而我们就只好自立更生了。

你可能注意到除了 `printf` 之外，还有一些用于 log 的彩色输出宏。感兴趣的同学可以看看 `log.h`。

- **sbi.c**

`printf` 的实现在 `printf.c`，在函数之中我们完成了对 `format` 字符串的解析工作。那么我们是如何把字符串真正地打印到 shell 上的呢？我们调用 `consputc` 函数输出一个 `char` 到 shell，而 `consputc` 函数其实就是调用了 `sbi.c` 之中的 `console_putchar` 函数。这个 `console_putchar` 函数的本质是调用了 `sbi_call`。剥开层层套娃，大家可以发现打印的最终实现是使用 `sbi` 帮助我们包装好的 `ecall` 汇编代码，通过指定 `ecall` 的 `idx` 为 `SBI_CONSOLE_PUTCHAR`，并将我们的字符做为参数传入到 `ecall` 指定的寄存器之中完成一次系统调用来实现的。本来，作为一个 OS，串口输出(也就是输出到 shell)的事情也应该我们自己来做，但这里为了简化这些硬件强相关的实现，我们利用 `rust-sbi` 的 M 态支持。这也是 `riscv` 灵活性的一个体现。

`rustsbi` 拓展阅读：[rsutsbi](#)。

bootloader文件夹

这个文件夹是用来存放 `bootloader`(也就是 `rustsbi`) 的 `bin` 文件的，这一章以及之后都无需我们做任何修改。

硬件加电之后是处于M态，而 `rustsbi` 帮助我们完成了 M 态的初始化，最终将 PC 移动至我们 os 开始执行的位置。同时，它也会帮助S态的 os 完成一些基本管理，详情可以看 `os/sbi.c` 文件。

makefile 和 qemu

本节导读

为了帮助大家进一步理解我们的项目的链接和编译的过程，这里简要介绍一下 makefile 的内容。

警告

注意，makefile 在整个实验过程中不可修改，否则可能导致 CI 无法通过！

makefile 内部

指定编译使用的工具

```
TOOLPREFIX = riscv64-unknown-elf-  
CC = $(TOOLPREFIX)gcc  
AS = $(TOOLPREFIX)gas  
LD = $(TOOLPREFIX)ld  
OBJCOPY = $(TOOLPREFIX)objcopy  
OBJDUMP = $(TOOLPREFIX)objdump  
GDB = $(TOOLPREFIX)gdb
```

这里makefile调用了大家设定好的PATH之中的riscv64工具链。如果没有设置好，那么之后的编译就会因为找不到这些文件而出错。

添加编译flag

```
CFLAGS = -Wall -Werror -O -fno-omit-frame-pointer -ggdb  
CFLAGS += -MD  
CFLAGS += -mmodel=medany  
CFLAGS += -ffreestanding -fno-common -nostdlib -mno-relax  
CFLAGS += -I.  
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-  
stack-protector)
```

比较需要注意的是我们设置了警告也会报错，因此大家写代码的时候最好避免 warning 的出现，这是良好的编程习惯。

设置编译目标

```
# 目录定义  
K = os  
BUILDDIR = build  
  
# .o 目标的确定，也就是 os 目录下所有的 .c 和 .s 都编译成 .o  
C_SRCS = $(wildcard $K/*.c)  
AS_SRCS = $(wildcard $K/*.s)  
C_OBJS = $(addprefix $(BUILDDIR)/, $(addsuffix .o, $(basename $(C_SRCS))))  
AS_OBJS = $(addprefix $(BUILDDIR)/, $(addsuffix .o, $(basename $(AS_SRCS))))  
OBJS = $(C_OBJS) $(AS_OBJS)  
  
# kernel 镜像由所有的 .o 按照 kernel.ld 链接而成
```

```
$(BUILDDIR)/kernel: $(OBJS) $(K)/kernel.ld
$(LD) $(LDFLAGS) -T kernel.ld -o kernel $(OBJS)
```

请同学们自行查阅并了解 `wildcard`、`addprefix`、`addsuffix`、`basename` 的意义。

运行 qemu

```
QEMU = qemu-system-riscv64
QEMUOPTS = "
    -nographic "
    -smp $(CPUS) "
    -machine virt "
    -bios $(BOOTLOADER) "
    -kernel kernel

run: $(BUILDDIR)/kernel
$(QEMU) $(QEMUOPTS)
```

这里和前面一致。大家不需要太关心qemu的更多细节，我们涉及它的操作已经在makefile和sbi之中处理了。

gdb 调试

```
# QEMU's gdb stub command line changed in 0.11
QEMUGDB = $(shell if $(QEMU) -help | grep -q '^gdb'; then "
    echo "-gdb tcp::1234"; "
    else echo "-s -p 1234"; fi)

debug: kernel .gdbinit
$(QEMU) $(QEMUOPTS) -s $(QEMUGDB) &
sleep 1
$(GDB)
```

使用 `make debug` 来使用 `gdb` 调试 `qemu`。程序自身执行的机制和直接 `make run` 一样。在解析 `bootloader` 的行为时可以使用 `gdb` 在其中添加断点来查看对应寄存器和内存的内容。`gdb` 的具体使用方法和汇编课程上一致。不熟悉的同学可以在训练章节查看到可能用到的 `gdb` 指令的简单用法，也十分推荐同学们自学一些基础的 `gdb` 使用方法，掌握 `gdb` 对本课程帮助很大。

LOG 支持

```
ifeq ($(LOG), error)
CFLAGS += -D LOG_LEVEL_ERROR
else ifeq ($(LOG), warn)
CFLAGS += -D LOG_LEVEL_WARN
else ifeq ($(LOG), info)
CFLAGS += -D LOG_LEVEL_INFO
else ifeq ($(LOG), debug)
CFLAGS += -D LOG_LEVEL_DEBUG
else ifeq ($(LOG), trace)
CFLAGS += -D LOG_LEVEL_TRACE
endif
```

我们的 log 等级选择是通过 -D 参数来实现的，这也是大家 `make run LOG=xxx` 的原理。从这里我们也可以看到 LOG 的可选值。